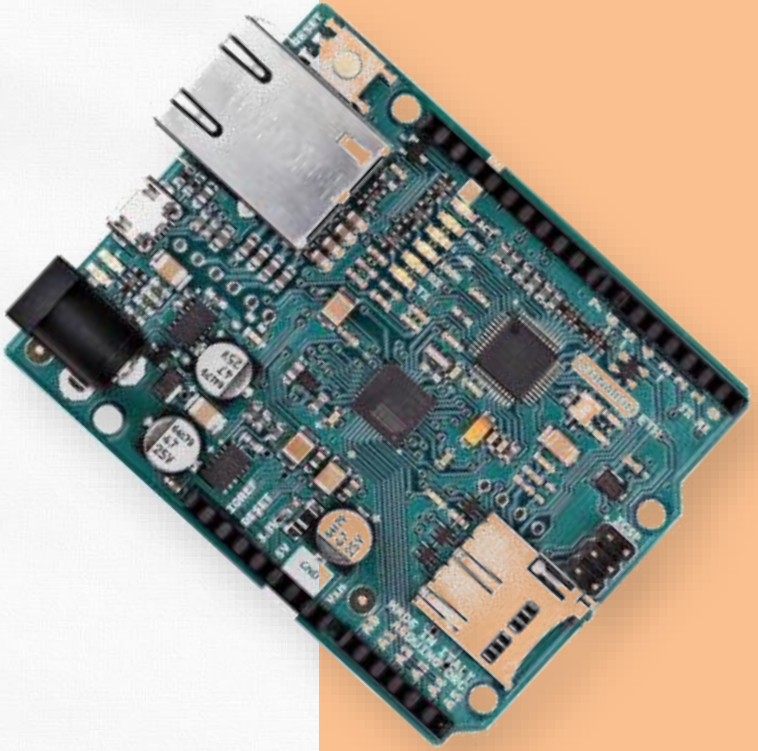
单片机



第6章：定时器/计数器

目 录

CONTENTS

01. 定时器概述

02. STM32的通用定时器

03. 定时器的HAL库接口函数

04. 定时器的HAL库应用实例

01

定时器概述

定时器的概念

能够实现定时功能的电路单元。

定时器本质：

定时器和计数器本质上都是计数器，定时器是计数器的一种特例，专门用于对周期信号计数。

定时器的功能

计数	定时	输入捕获	输出比较
脉冲计数，使用微控制器内部的外部时钟（PCLK）来计数，是对固定周期的脉冲信号计数。	时间控制，通过对微控制器内部的时钟脉冲进行计数实现定时功能。	对输入信号进行捕获，实现对脉冲的频率测量，可用于对外部输入信号脉冲宽度的测量，比如测量电机转速。	将计数器计数值和设定值进行比较，根据比较结果输出不同电平，用于控制输出波形，比如直流电机的调速。

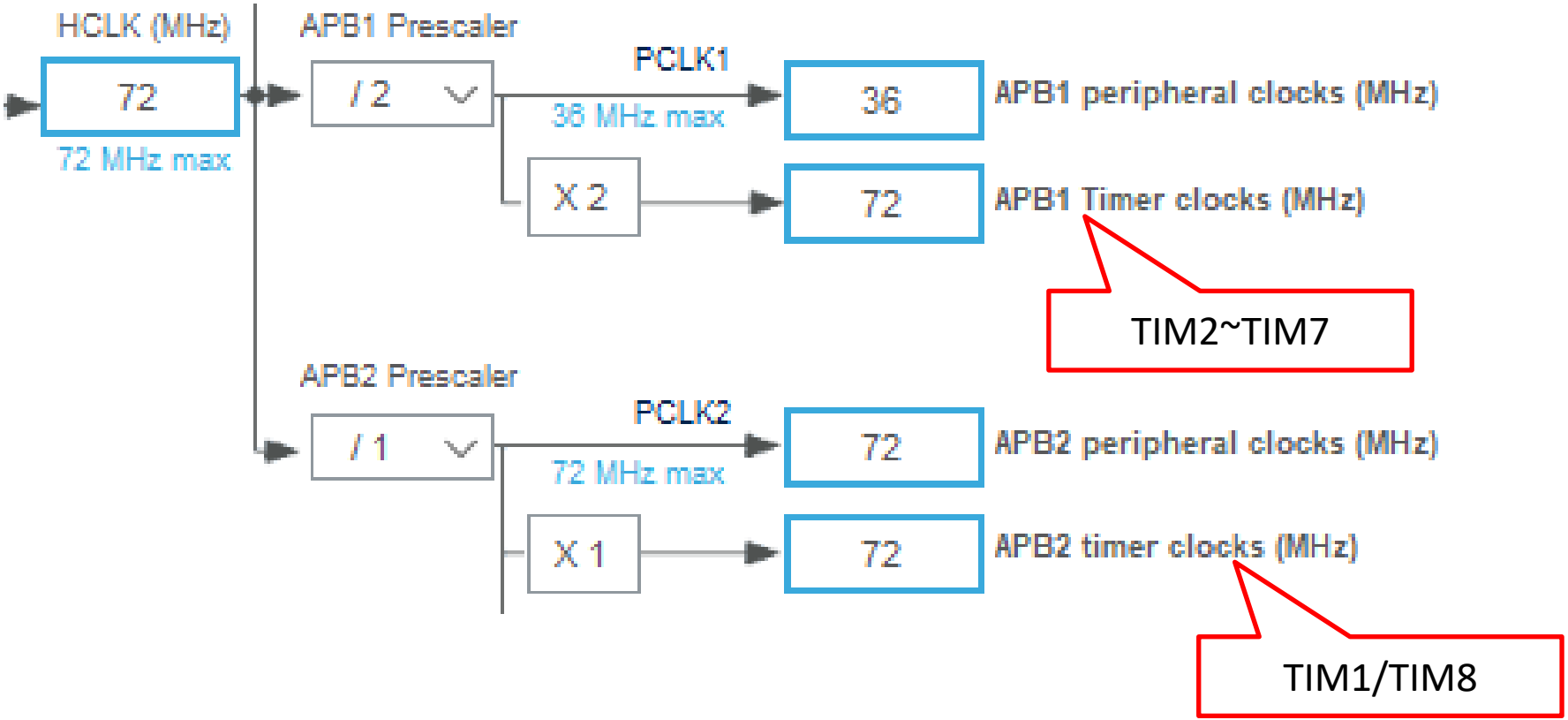
定时器的分类

- STM32定时器家族
  - 外设定时器
    - 常规定时器
      - 基本定时器
      - 通用定时器
      - 高级定时器
    - 专用定时器
      - 看门狗定时器
      - 实时时钟
      - 低功耗定时器
  - 内核定时器

定时器的分类

类型	名称	位数	计数方式	DMA请求	捕获/比较通道	定时时钟	挂接总线
高级定时器	TIM1	16位	向上/向下/中心对齐	可以	4	72MHz	APB2
	TIM8	16位		可以	4		
通用定时器	TIM2	16位	向上/向下/中心对齐	可以	4		APB1
	TIM3	16位		可以	4		
	TIM4	16位		可以	4		
	TIM5	16位		可以	4		
基本定时器	TIM6	16位	向上	可以	0		APB1
	TIM7	16位		可以	0		

定时器的时钟


HCLK (MHz)
72
72 MHz max
APB1 Prescaler
/ 2
PCLK1
36 MHz max
36
APB1 peripheral clocks (MHz)
X 2
72
APB1 Timer clocks (MHz)
TIM2~TIM7
APB2 Prescaler
/ 1
PCLK2
72 MHz max
72
APB2 peripheral clocks (MHz)
X 1
72
APB2 timer clocks (MHz)
TIM1/TIM8

02

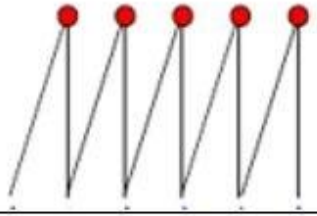
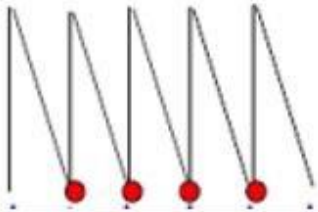
STM32的通用定时器

定时器的计数模式

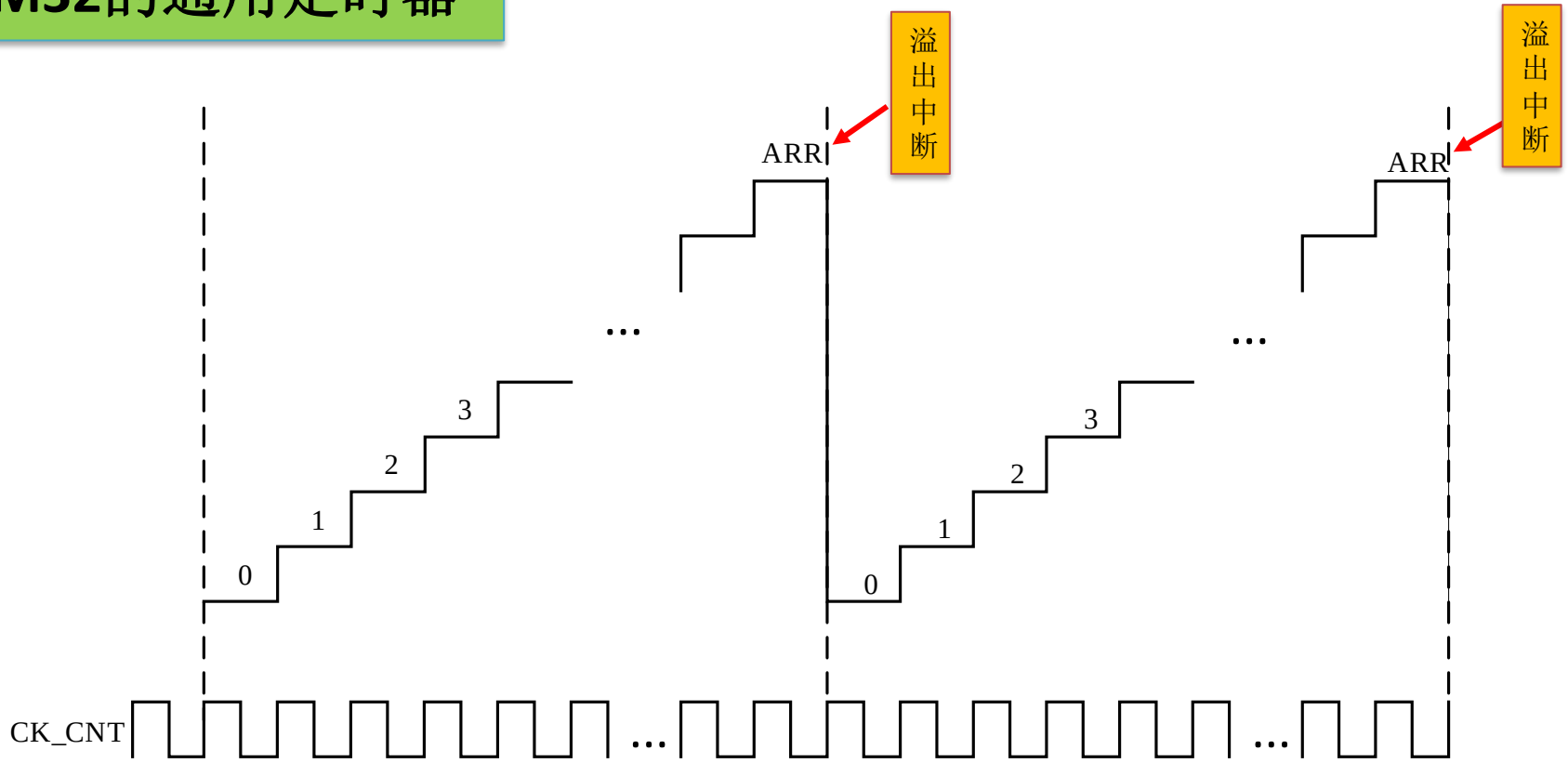
- 通用定时器TIM2~TIM5的计数模式有：**向上计数（递增）、向下计数（递减）、向上向下双向计数（中心对齐）**。
- **向上计数模式：**计数器从 0 计数到自动加载值(TIMx_ARR)，然后重新从 0 开始计数并且产生一个计数器溢出事件。
- **向下计数模式：**计数器从自动装入的值(TIMx_ARR)开始向下计数到 0，然后从自动装入的值重新开始，并产生一个计数器向下溢出事件。
- **中心对齐模式：**计数器从 0 开始计数到自动装入的值ARR-1，产生一个计数器溢出事件，然后向下计数到 1 并且产生一个计数器溢出事件；然后再从 0 开始重新计数。

STM32的通用定时器

定时器的计数模式

计数模式	计数示意图	计数过程
向上计数模式		从 0 开始递增计数，直到 ARR，发生溢出。计数器重装为 0，开始下一轮计数。
向下计数模式		从 ARR 开始递减计数，直到 0，发生溢出。计数器重装为 ARR，开始下一轮计数。
中心对齐计数模式		从 0 开始递增计数，记到 ARR-1，发生溢出；计数器重装为 ARR，然后开始向下计数直到 1 时发生溢出；然后计数器重装为 0，开始下一轮计数。
注意各个计数模式的计数器起始值、重装值、溢出点。		

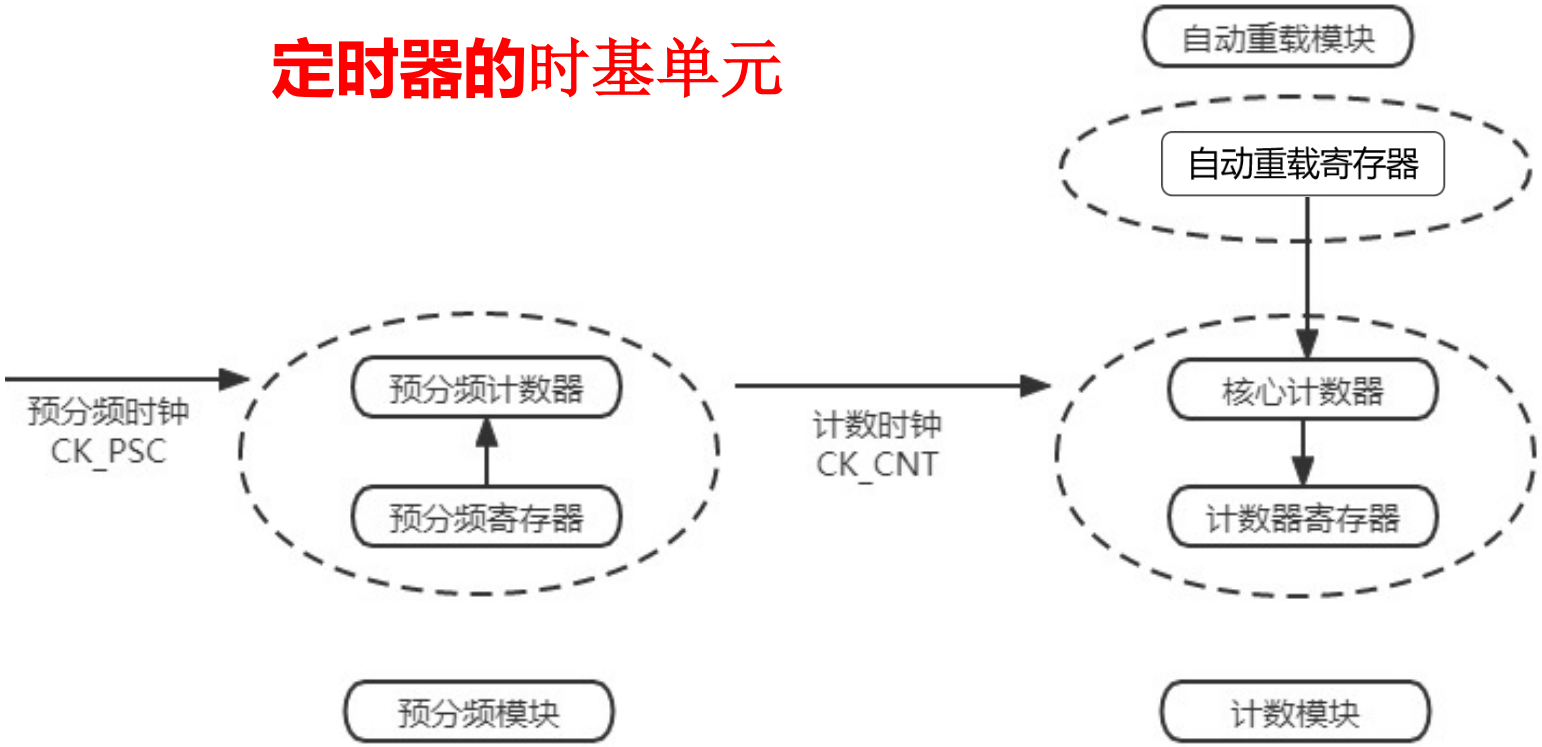
STM32的通用定时器



思考：通用定时器超出(溢出)时间多长？ 由哪些寄存器参数决定？

STM32的通用定时器

定时器的时基单元



STM32的通用定时器

预分频模块由预分频计数器和预分频寄存器组成，预分频计数器通过计数的方式对预分频时钟进行分频，寄存器用来存放预分频系数PSC。由此可得计数时钟CK_CNT与预分频时钟CK_PSC的关系式：

$$CK_CNT=CK_PSC/(PSC+1)$$

计数模块由核心计数器和计数寄存器组成，计数器对时钟信号CK_CNT进行计数，每输入一个脉冲信号，计数器的值就会自动加一或减一。

自动重载模块由自动重载寄存器（TIMx_ARR）组成，用来设置计数器的终值或初值，决定计数脉冲的多少或定时周期的长短，当定时器为16位时，其取值范围为0~65535（$2^{16}-1$）。

定时时间的计算

计数时钟CK_CNT与预分频时钟CK_PSC的关系式：

$$CK_CNT = CK_PSC / (PSC+1)$$

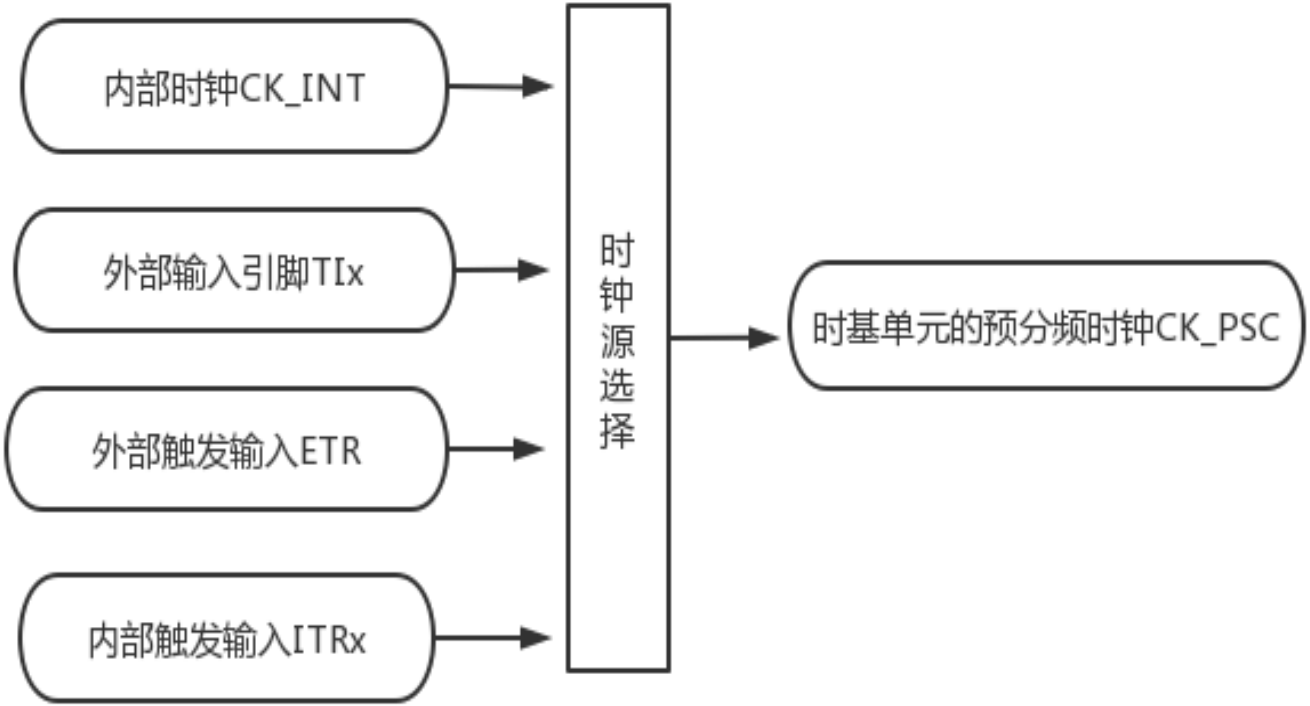
则定时时间T = （ARR+1）*（PSC+1）/CK_PSC

则**定时器的溢出频率**f = CK_PSC/ [（ARR+1）*（PSC+1）

]

STM32的通用定时器

定时器的时钟源



定时时间的计算

（1）假设定时器的时钟为72MHz，预分频系数为71，自动重载值为999。

$$T = (999+1)*(71+1)/72000000 = 1\text{ms}$$

可得定时时间为：

$$f = 72000000/((999+1)*(71+1)) = 1000\text{Hz}$$

可得定时器的溢出频率为：

（2）假如要设置定时时间为500ms，并配置中断，CK_INT按照系统默认初始化来（即72MHz），PSC取7199，则ARR取值应为多少？

STM32的通用定时器

定时器相关寄存器

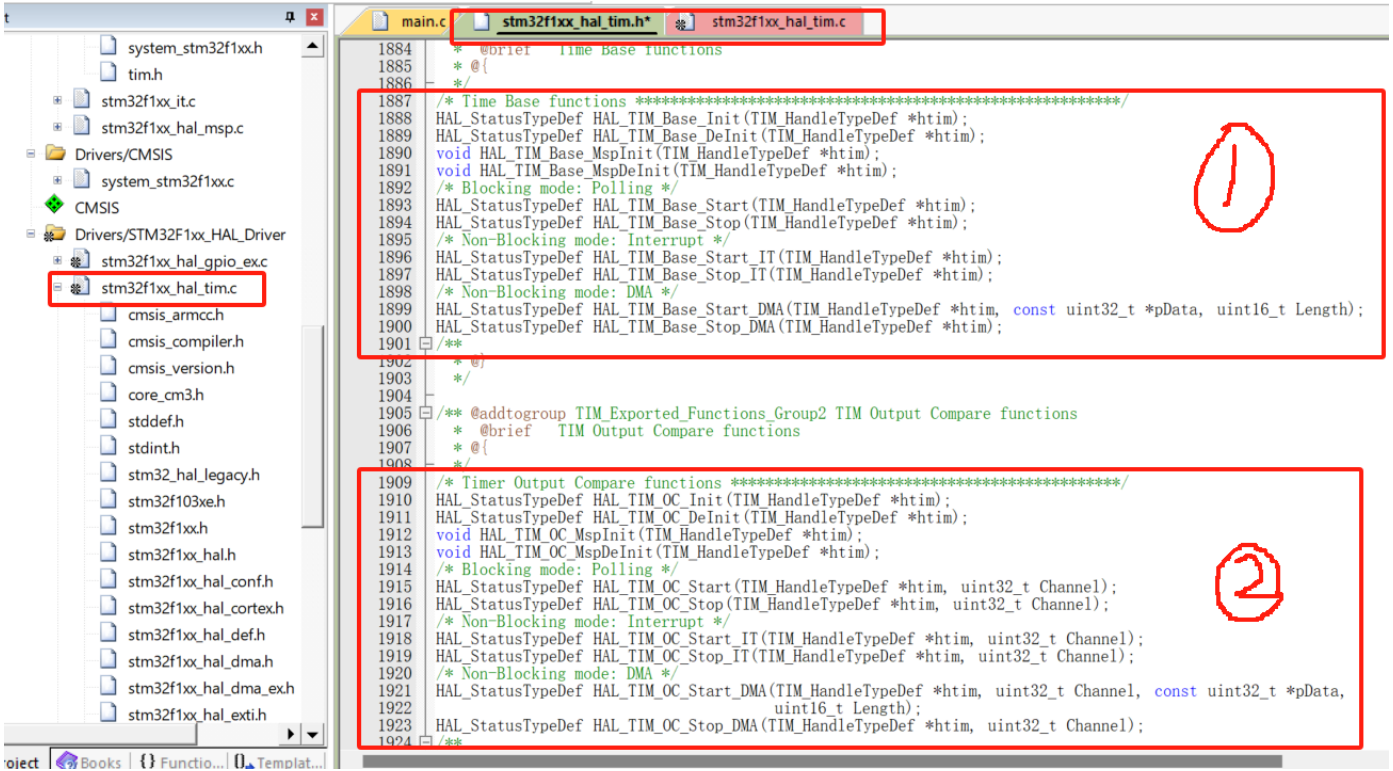
- 控制寄存器（TIMx_CR）
- DMA/中断使能寄存器（TIMx_DIER）
- 预分频寄存器（TIMx_PSC）
- 计数器（TIMx_CNT）
- 自动重装载寄存器（TIMx_ARR）
- 状态寄存器（TIMx_SR）

HAL库函数已封装好各类寄存器。通过调用相关的库函数即可配置寄存器。

03

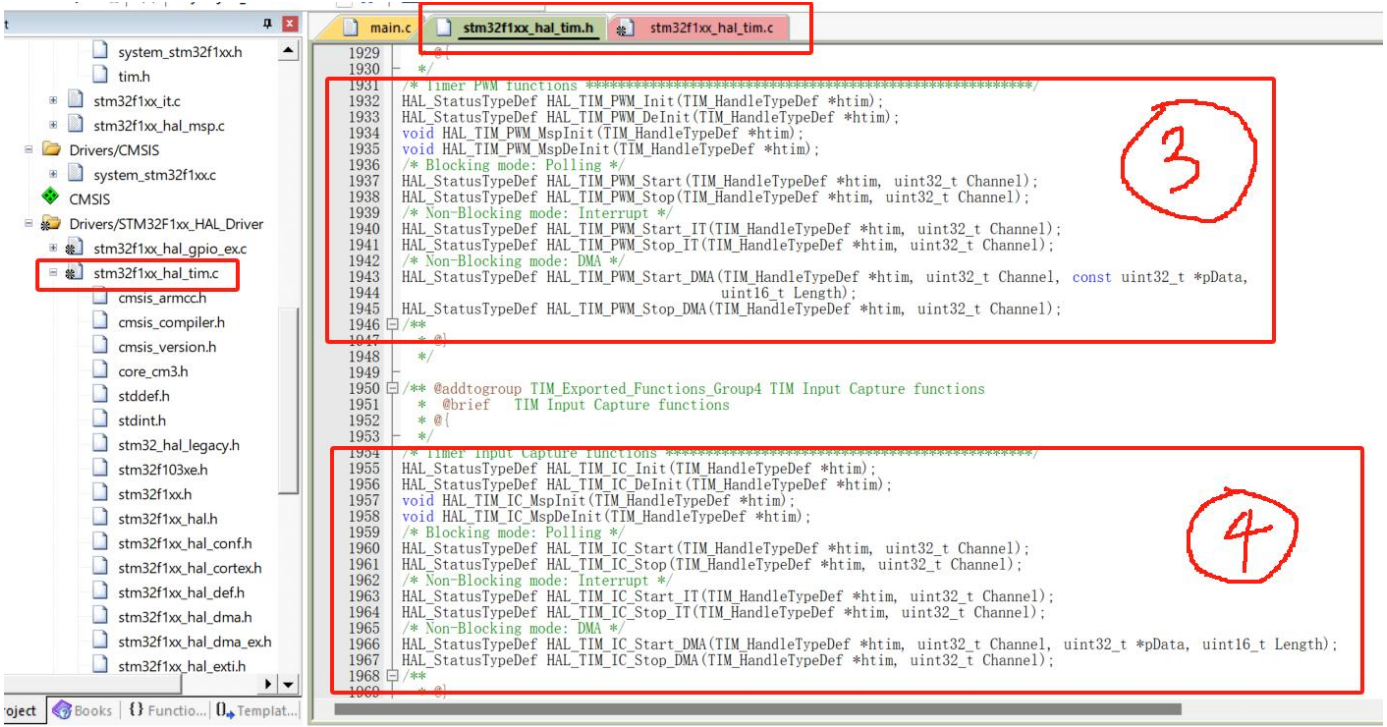
定时器的HAL库接口函数

定时器的HAL库接口函数



定时器的HAL库接口函数的源码在源文件stm32f1xx_hal_tim.c中，其对应的头文件stm32f1xx_hal_tim.h声明了TIM所有的库函数。

定时器的HAL库接口函数



定时器的HAL库接口函数的源码在源文件stm32f1xx_hal_tim.c中，其对应的头文件stm32f1xx_hal_tim.h声明了TIM所有的库函数。

定时器的HAL库接口函数

1、定时器初始化函数

项目	函数说明
函数原型	**HAL_TIM_Base_Init(TIM_HandleTypeDef *htim)**
功能描述	根据定时器句柄中设定的参数初始化时基单元
入口参数	**huart：为结构体类型TIM_HandleTypeDef定义的指针，取值为huart1 ~ huart8**
返回值	**HAL_OK：初始化成功HAL_ERROR：初始化失败**
注意事项	定时器初始化函数通过调用底层初始化函数**HAL_TIM_Base_MspInit(htim)**完成引脚、时钟和中断的设置，该函数由**STM32CubeMX**软件自动生成。
参考示例	HAL_TIM_Base_Init(&htim3)　　//初始化定时器3

2、定时器轮询方式启动函数

项目	函数说明
函数原型	**HAL_StatusTypeDef HAL_TIM_Base_Start (TIM_HandleTypeDef *htim)**
功能描述	在轮询方式下启动定时器
入口参数	**huart：为结构体类型TIM_HandleTypeDef定义的指针，取值为huart1 ~ huart8**
返回值	**HAL_OK：启动成功HAL_ERROR：启动失败**
注意事项	该函数需在定时器初始化完成之后由用户调用，以使能相应的定时器。
参考示例	HAL_TIM_Base_Start(&htim1) //轮询方式开启定时器1

3、定时器中断方式启动函数

项目	函数说明
函数原型	**HAL_StatusTypeDef HAL_TIM_Base_Start_IT(TIM_HandleTypeDef* htim)**
功能描述	在中断方式下启动定时器
入口参数	**huart：为结构体类型TIM_HandleTypeDef定义的指针，取值为huart1 ~ huart8**
返回值	**HAL_OK：启动成功HAL_ERROR：启动失败**
注意事项	该函数需在定时器初始化完成之后由用户调用该函数使能定时器； 中断启动前需清除在时基单元初始化过程中产生的中断标志。
参考示例	HAL_TIM_Base_Start_IT(&htim8) //中断方式开启定时器8

4、定时器中断通用函数

接口函数：HAL_TIM_IRQHandler	
函数原型	void HAL_TIM_IRQHandler (TIM_HandleTypeDef * htim)
功能描述	所有定时器中断发生后的通用处理函数
入口参数	Htim：指向定时器句柄的指针（即定时器句柄的地址）
返回值	无
注意事项	该函数由STM32CubeMX软件自动生成。
参考示例	HAL_TIM_IRQHandler(&htim3)

5、定时器更新中断回调函数

接口函数：**HAL_TIM_PeriodElapsedCallback**	
函数原型	HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
功能描述	回调函数，用来处理具体的中断任务
入口参数	htim：指向定时器句柄的指针（即定时器句柄的地址）
返回值	无
注意事项	由定时器中断通用处理函数（**HAL_TIM_IRQHandler**）调用； 由用户根据实际任务进行程序编写。

6、定时器中断标志清除函数

接口函数：__HAL_TIM_CLEAR_IT	
宏原型	__HAL_TIM_CLEAR_IT(__HANDLE__, __INTERRUPT__)
功能描述	清除定时器的各类中断标志
宏参数1	__HANDLE__：指向定时器句柄的指针（即定时器句柄的地址）
宏参数2	__INTERRUPT__：定时器的中断标志，常用取值如下： TIM_IT_UPDATE：更新中断标志 TIM_IT_CC1：通道1的捕获/比较中断标志 TIM_IT_CC2：通道2的捕获/比较中断标志 TIM_IT_CC3：通道3的捕获/比较中断标志 TIM_IT_CC4：通道4的捕获/比较中断标志
返回值	无
注意事项	由用户调用

7、定时器计数值读取函数

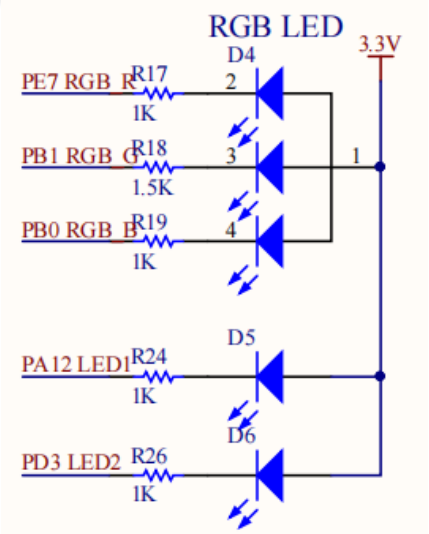
接口函数：__HAL_TIM_GET_COUNTER	
宏原型	__HAL_TIM_GET_COUNTER(__HANDLE__)
功能描述	获得定时器的计数值
宏参数	__HANDLE__：指向定时器句柄的指针（即定时器句柄的地址）
返回值	定时器的计数值
注意事项	由用户调用
参考示例	__HAL_TIM_GET_COUNTER(&htim3) //读取定时器3的当前计数值

04

定时器的HAL库应用实例

定时器HAL库编程实践一

硬件设计



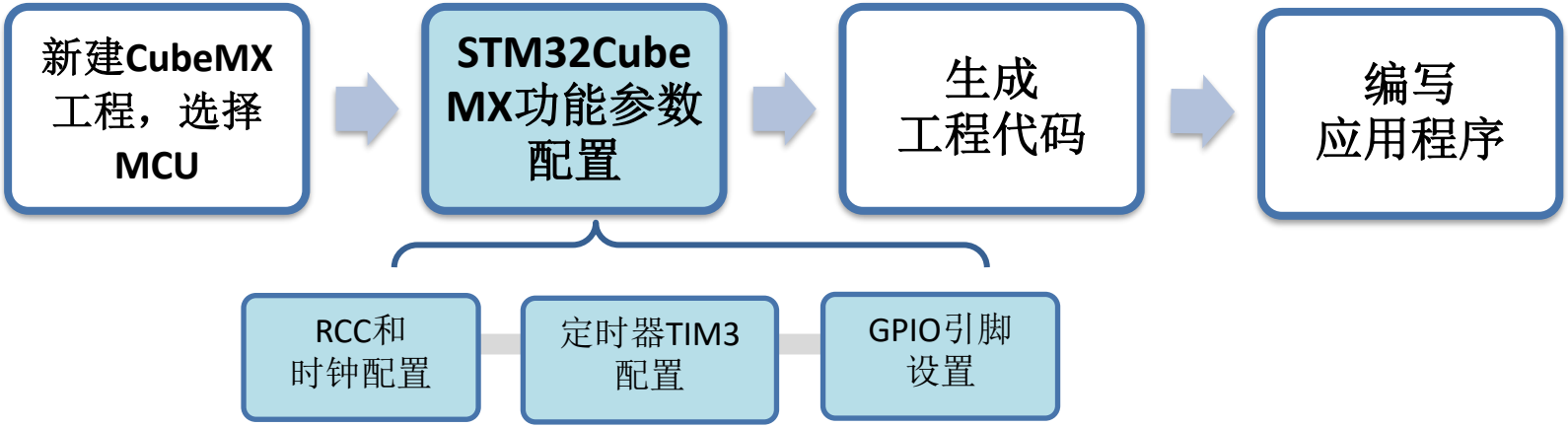
功能

利用基于HAL库采用定时器TIM3产生精确延时1s

利用定时器TIM3产生1s的定时中断，然后在中断回调函数中控制LED1灯，实现LED1的状态取反（闪烁）。LED1连接在STM32芯片的PA12引脚上，低电平有效

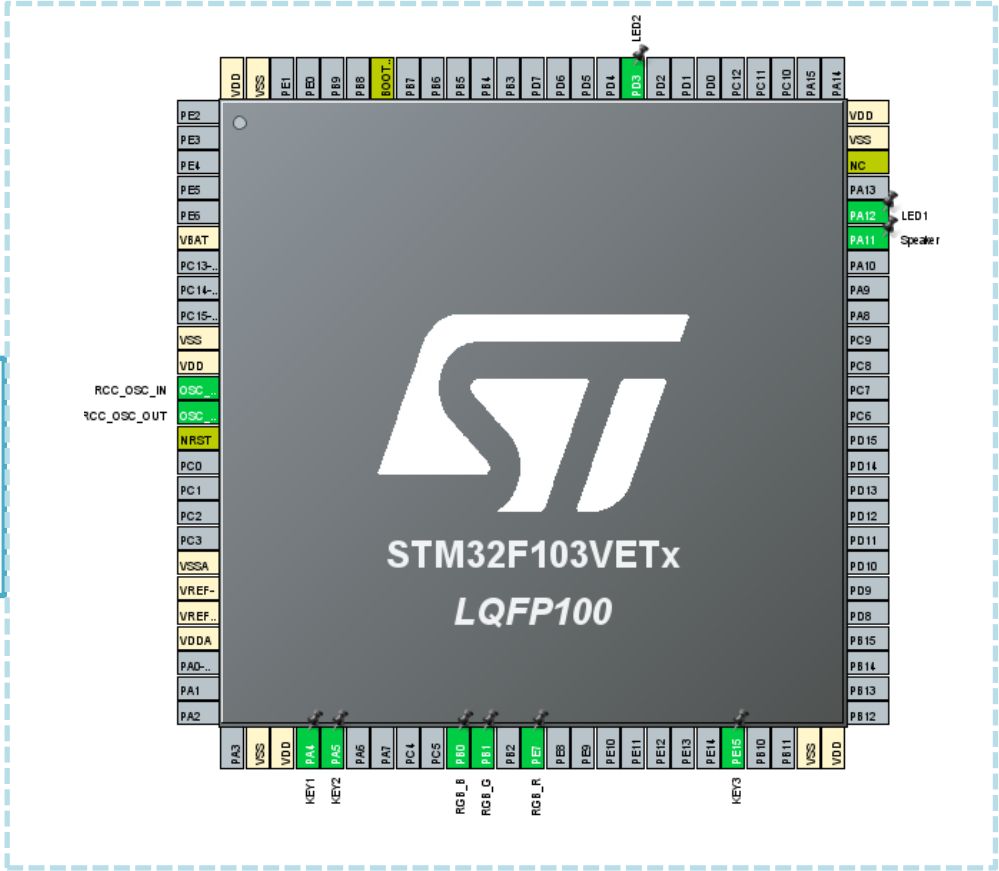
定时器HAL库编程实践一

软件设计流程



软件设计—新建STM32CubeMX工程，选择MCU

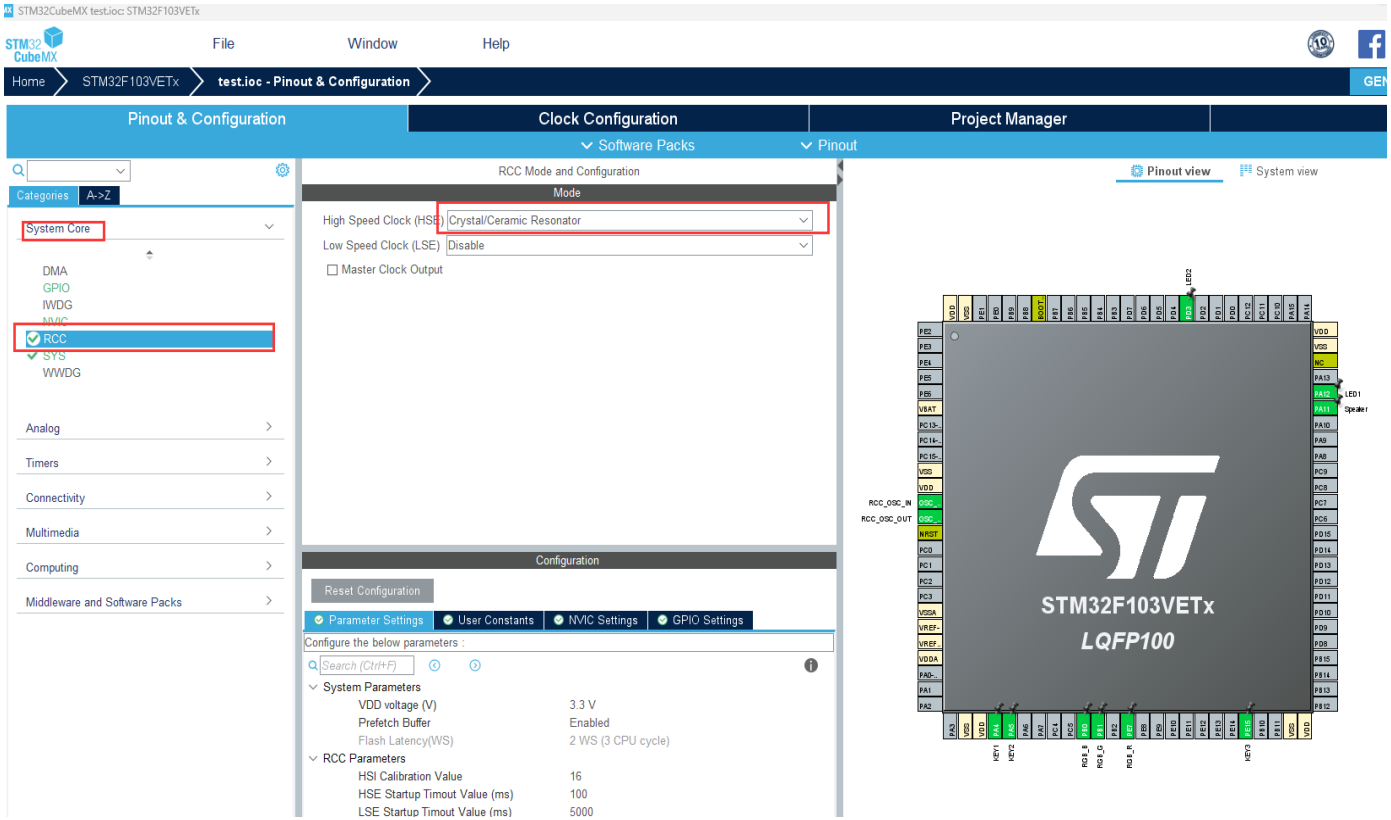
新建一个CubeMX工程，选择MCU，这里采用STM32F103VET6芯片



软件设计—STM32CubeMX功能参数配置

RCC配置

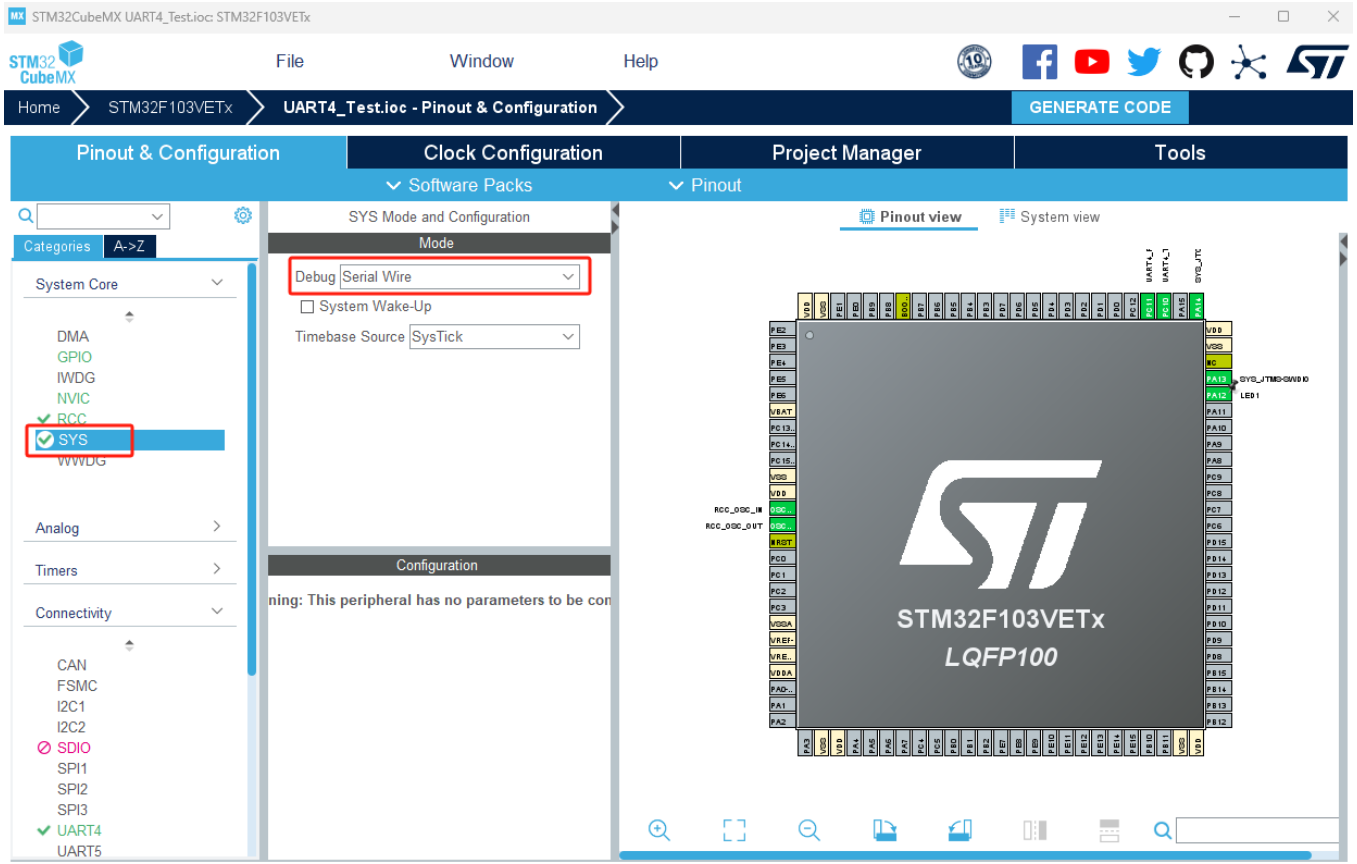
HSE选择“Crystal/Ceramic Resonator”（晶振/陶瓷谐振器），LSE选择“Disable”



软件设计—STM32CubeMX功能参数配置

SYS配置

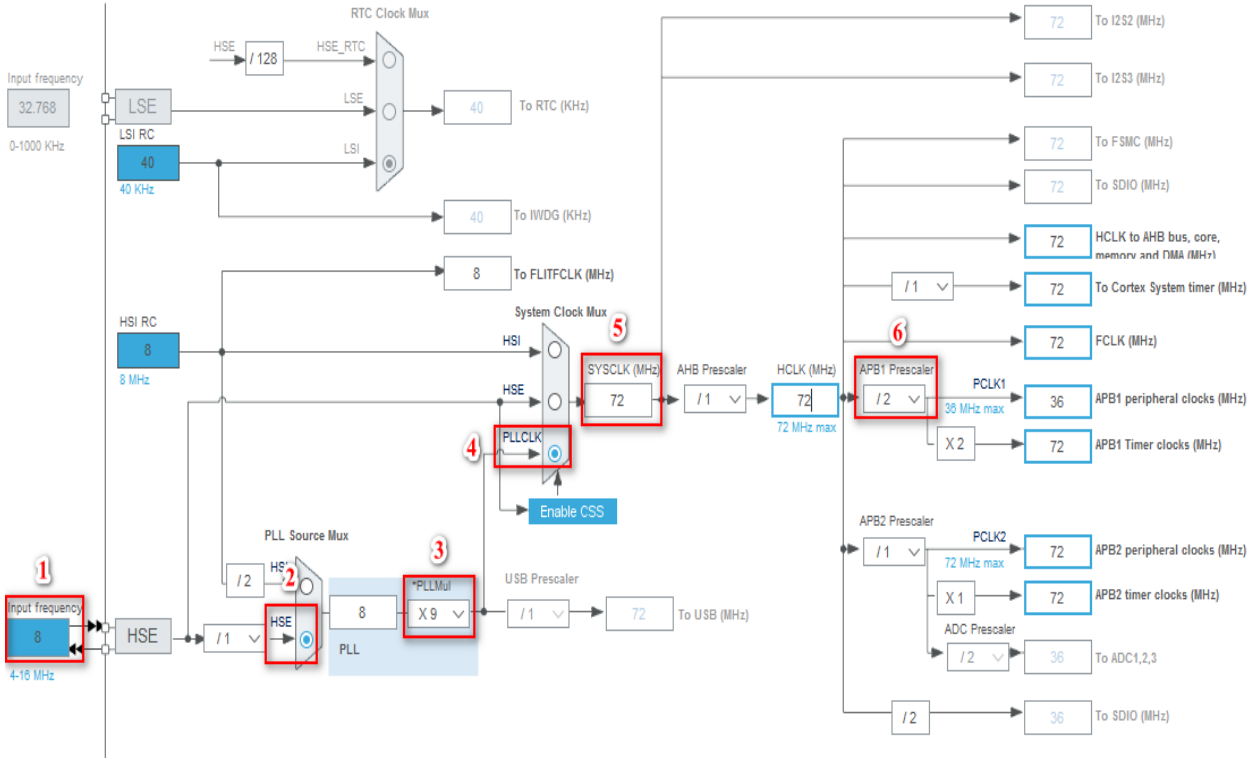
SYS模块用于配置程序的调试接口，这里Debug选择”Serial Wire”



软件设计—STM32CubeMX功能参数配置

时钟配置

配置STM32的时钟系统，系统时钟配置为72MHz，APB2为72MHz，配置APB1为36MHz。

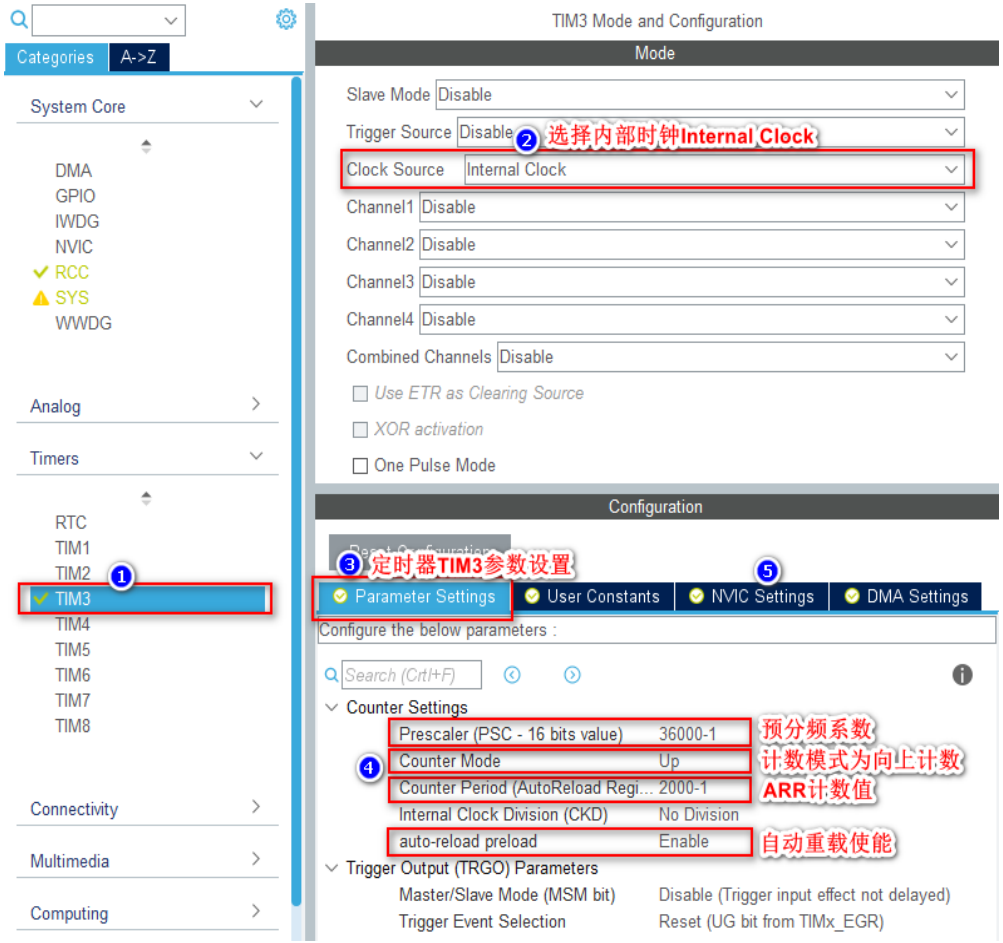


软件设计—STM32CubeMX功能参数配置

定时器TIM3配置

- “Clock Source”设置为“Internal Clock”（内部时钟）
- 预分频系数“Prescaler”设置为“36000-1”；“Counter Mode”设置为“Up”
- 计数值“Counter Period”设置为“2000-1”
- “auto-reload preload”设置为“Enable”

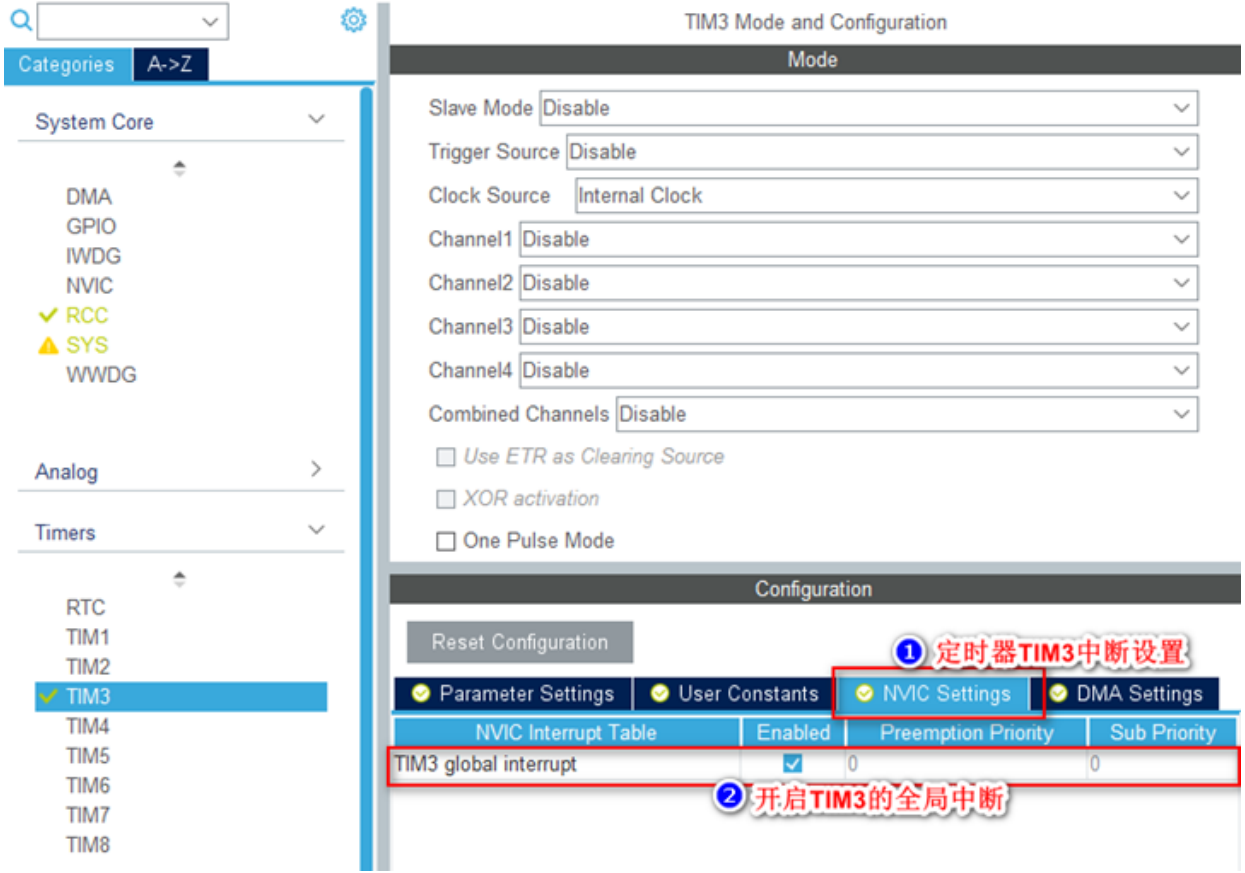
则定时器TIM3精确定时1s



软件设计—STM32CubeMX功能参数配置

定时器TIM3配置

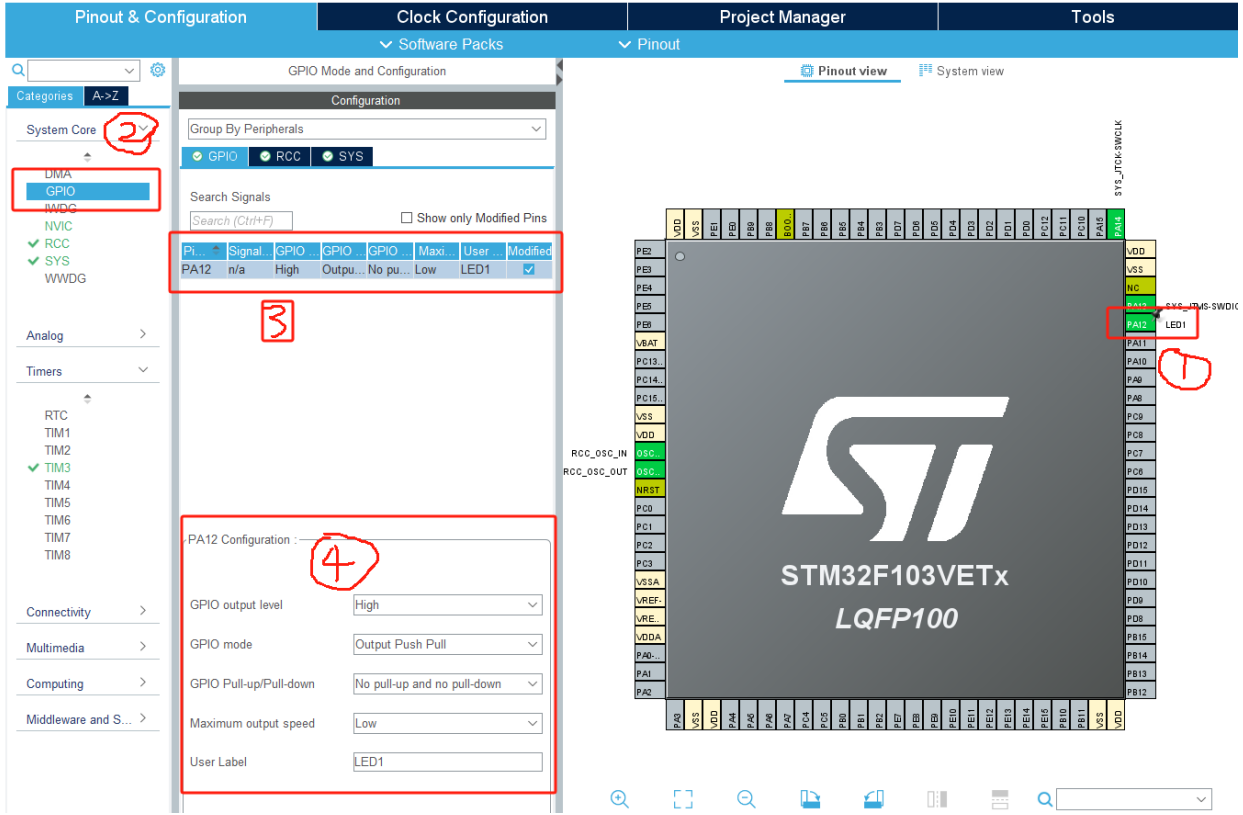
- 在定时器TIM3参数配置“Configuration”选项配置页的“NVIC Settings”中使能定时器TIM3的全局中断



软件设计—STM32CubeMX功能参数配置

GPIO引脚设置

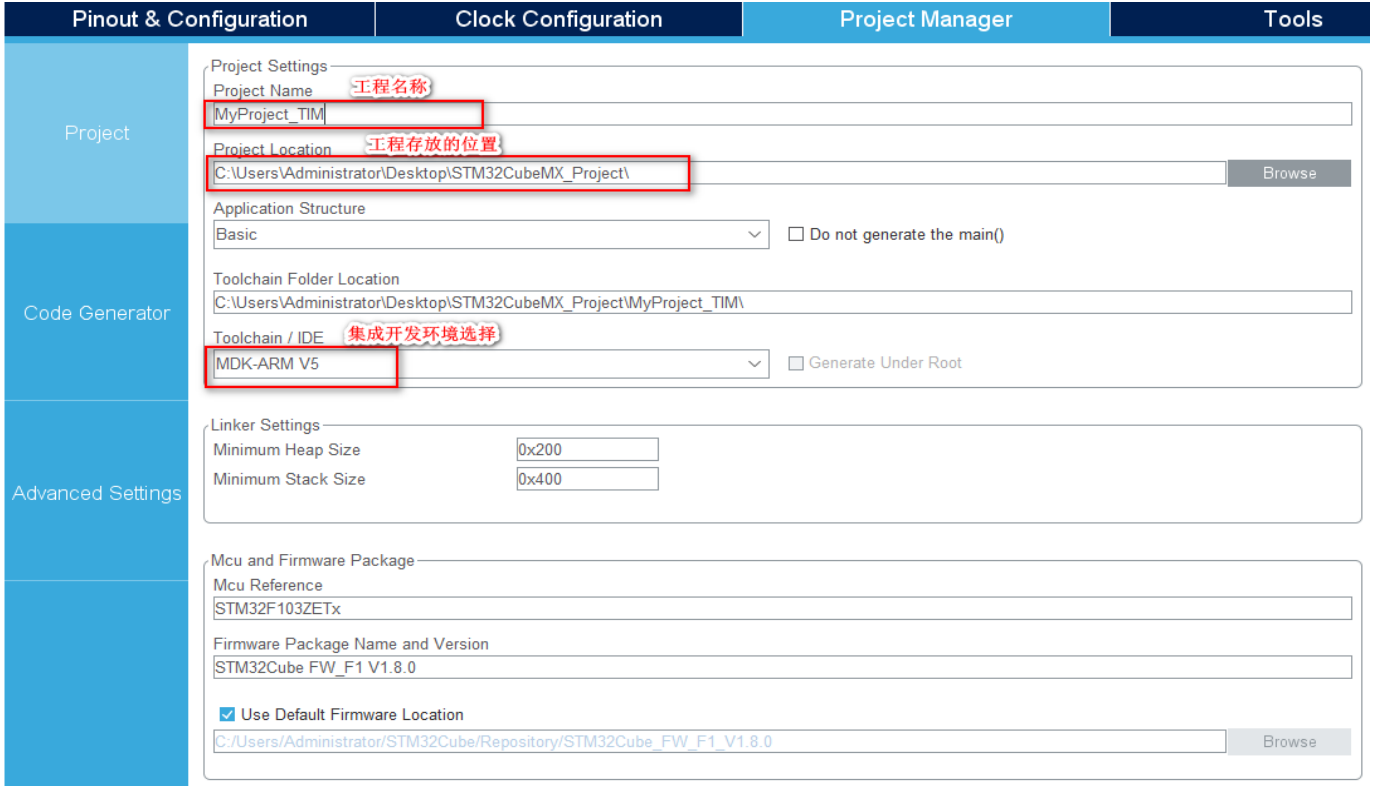
- 设置PA12引脚为“GPIO_Output”，用于本例的LED指示灯显示，并在“GPIO Mode and Configuration”中配置相应的参数



软件设计—生成工程代码

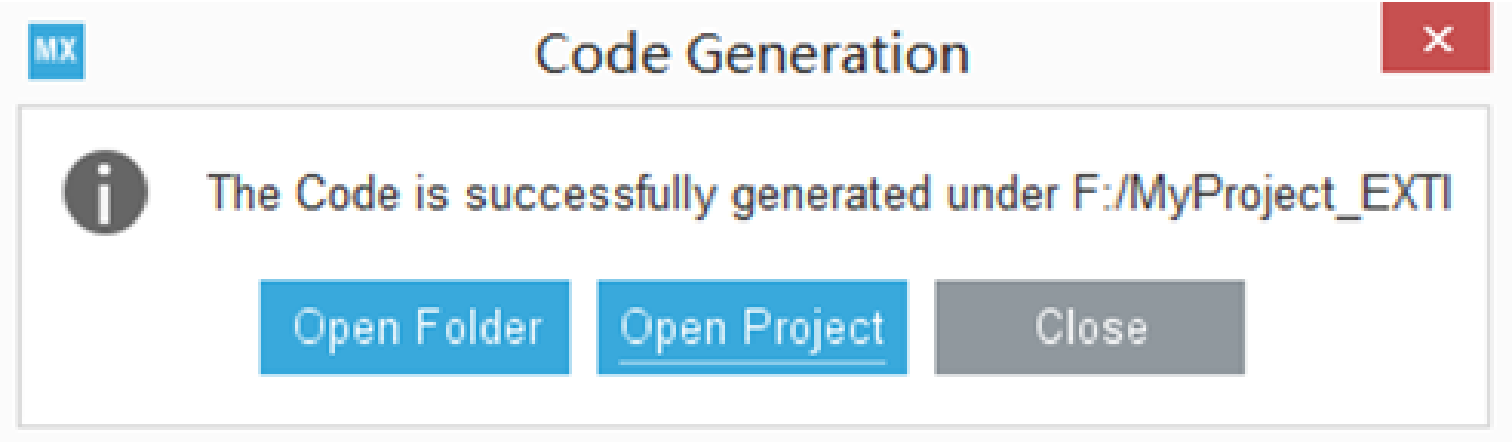
工程相关参数配置

配置工程名称、工程保存位置、选择“MDK-ARM V5”编译器等，生成工程代码



软件设计—生成工程代码

通过STM32CubeMX的菜单栏中的“Generate Code”生成工程代码，生成代码后，会提示是否打开该工程窗口



软件设计—编写用户程序

在main.c文件中的/*USER CODE BEGIN2 */和/*USER CODE END2 */之间添加开启定时器TIM3中断的程序

```
MX_TIM3_Init();

/* USER CODE BEGIN 2 */

HAL_TIM_Base_Start_IT(&htim3);

//启动定时器TIM3定时中断

/* USER CODE END 2 */
```

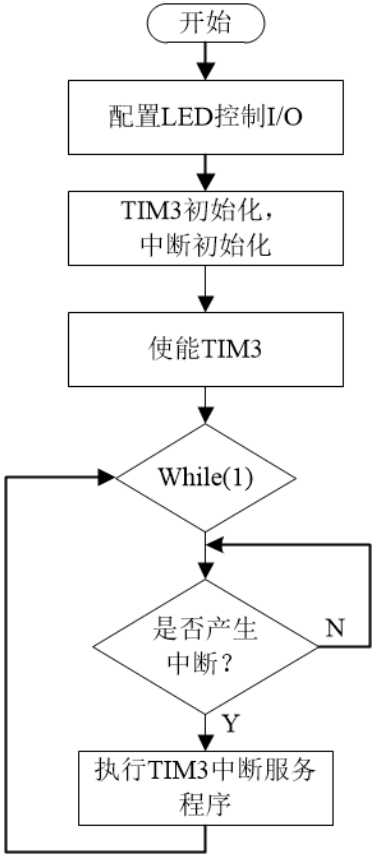
软件设计—编写用户程序

在mian.c文件的/*USER CODE BEGIN4 */和/*USER CODE END4 */之间添加TIM3的中断回调函数

```
/* USER CODE BEGIN 4 */
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if(htim->Instance == htim3.Instance)
        {
            HAL_GPIO_TogglePin(GPIOA,GPIO_PIN_12);
        }
}
/* USER CODE END 4 */
```

定时器HAL库编程实践一

定时器TIM3中断实验程序流程图



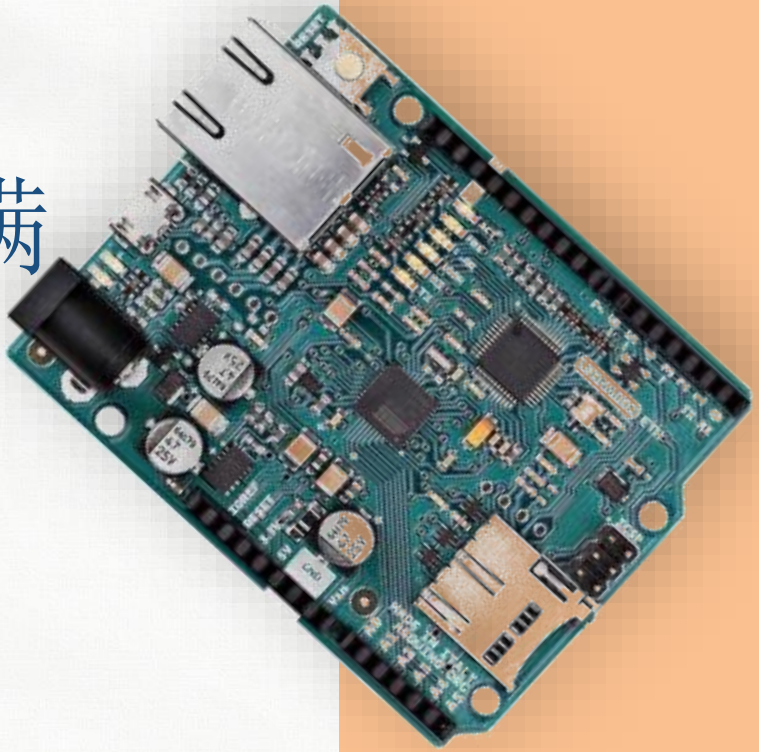
定时器中断主程序流程图

定时器HAL库编程实践二

任务一：

通过定时器TIM1产生500ms的定时中断（即每500ms中断一次），然后在中断回调函数中控制LED2灯，实现LED2的状态取反（闪烁）。

任务二：

自定义延时函数，采用TIM5定时器实现精确的延时，实现开发板上的RGB灯按红、绿、蓝灯顺序以500ms的间隔循环闪烁。即用自定义的延时函数来代替HAL_Delay( )延时函数。

本章结束，收获满满